

UNDERSTANDING LOOTVAULT

1 Your application and its boundaries

This guide explains your actual project from a React click to a committed PostgreSQL transaction. React presents the game, Spring Boot decides the game, and PostgreSQL preserves the result. You should be able to trace a request, explain a coin change, and decide whether a proposed change needs a migration.

Your stack

React 19 and React Router render pages. Vite 8 serves and builds the frontend. Java 21 and Spring Boot 4.1 run the API. Spring Security manages authentication and CSRF. JPA with Hibernate maps entities to tables; JdbcTemplate handles explicit SQL. PostgreSQL stores durable state. Flyway applies schema changes. Maven builds Java; npm manages frontend dependencies.

Layer	Owns	Must not decide
React	Layout, feedback, local preferences	Winning hands or free coins
Spring Boot	Rules, identity, transactions	Whether an animation looked exciting
PostgreSQL	Committed balances and ownership	Page layout
Flyway	Ordered database structure changes	Live random outcomes

Three kinds of state

Local presentation state includes a tutorial step, an inventory filter, and an open cutscene. Shared frontend state includes the signed-in player and displayed wallet. Durable state includes owned items, balances, claims, saved hands, and tutorial completion. Refreshing can erase local state; it must not erase durable state.

Your first reading path

Start with `lootvault-frontend/src/App.jsx` for routes, `api/client.js` for HTTP, and `components/AppShell.jsx` for navigation. Then read `EarnLootController.java`, `EarnLootService.java`, and `CardGameEngine.java`. Backend class paths below are relative to `src/main/java/com/example/lootvaultproject`, unless stated otherwise.

Read this guide alongside the files. Before accepting a generated change, explain its input, its output, what it stores, and what happens if it fails. That habit is more valuable than recognizing every framework annotation immediately.

2 How a click reaches the server

Claiming the daily coin crate is a useful request to trace because its expected balance change is easy to check.

The browser starts the request

DailyCoinCrate calls `claimDailyCoins` from `api/earn.js`. That function calls `api.post("/api/earn/daily/claim")`. The shared client obtains `/api/auth/csrf` when needed, sends the session cookie, and adds the server-specified CSRF header. A CSRF rejection can refresh once; a network failure never blindly replays a write.

The development proxy

The browser opens Vite on `localhost:5173` or `5174`. `vite.config.js` forwards `/api` to Spring Boot on `localhost:8081`. The browser continues to see its frontend origin. `localhost` and `127.0.0.1` are different hostnames; choose one for a session. A fallback Vite port is also a different origin.

Step	File or component
1	DailyCoinCrate.jsx
2	api/earn.js and api/client.js
3	Vite proxy or same-origin production host
4	SecurityConfig and EarnLootController
5	EarnLootService and WalletService
6	PostgreSQL, then EarnResponse JSON
7	Component state and WalletContext

The response returns

Security checks run before the controller. The controller resolves the player from the authenticated identity; the browser does not choose a player ID. `EarnLootService` performs the claim transactionally and returns an `EarnResponse` DTO. Spring serializes it to JSON.

The component stores that response and refreshes `WalletContext`. Gameplay writes also emit `lootvault:wallet-changed`. The header reads the same context, avoiding a separate stale balance. React rendering changes the visible page; it does not directly update PostgreSQL.

Debug in that order

Inspect the Network panel: URL, method, status, and response. Then inspect the controller route, service rule, and database row or exception. A cutscene proves that an animation ran. The response and durable rows prove that a payout committed.

3 Login sessions and CSRF

A session cookie identifies a server-side login. A CSRF token protects a write made through that cookie. They solve different problems.

Registration and authentication

AuthService.registerPlayer checks uniqueness, BCrypt-hashes the password, creates a player and wallet, and records the 500-coin signup entry in one transaction. Registration does not log in; AuthContext logs in afterward. Plaintext passwords do not belong in storage, logs, or Git.

AuthController asks AuthenticationManager to authenticate credentials. It creates a SecurityContext, changes an existing session ID, and stores the context in the HTTP session. The cookie points to that session; it contains neither the password nor a balance. The client uses credentials: include to send it.

Secure writes

GET /api/auth/csrf returns a session-bound token. The client adds its header on POST. Login and developer login clear the previous token; logout invalidates the session. The client clears its token cache after authentication writes. CSRF_INVALID is emitted before a gameplay controller runs, allowing one safe token refresh and retry.

Response	Meaning	Check
204 from auth me	Signed out normally	Show login
401	Credentials or session missing	Cookie and auth response
403 CSRF_INVALID	Missing or stale write token	Header and session
403 Invalid CORS request	Origin not allowed	Host and frontend port
403 permission message	Forbidden action	Route authorization

Expiry and recovery

GET /api/auth/me returns 204 when no session exists. Private APIs return 401. The frontend clears auth state and remembers the route to return to after sign-in. A generation counter prevents a slow initial session lookup from overwriting a newer login.

Production uses secure HttpOnly cookies, SameSite=Lax, and a 30-minute idle timeout. HTTPS is required. Sessions currently live in application memory, so a server restart signs users out. Saved hands remain in PostgreSQL. Multiple replicas need shared sessions or a deliberate routing strategy.

4 How Spring Boot assembles the backend

Spring creates application objects and connects dependencies so controllers can use services without constructing them by hand.

Startup and dependency injection

LootVaultProjectApplication starts scanning. `@RestController` classes become HTTP entry points. `@Service` classes hold business rules. Spring Data provides implementations for `@Repository` interfaces. Constructor parameters declare dependencies that Spring supplies. A missing dependency or circular dependency normally fails startup.

The division of work

A controller maps a method and URL, reads a DTO, resolves the player, and returns a DTO. `EarnLootController.Start` contains the request ID, game, and stake. `EarnLootService` checks the actual minimum and balance. A disabled React button is useful feedback, but it is not API security.

`WalletService` handles coins and the ledger. `CrateService` consumes crates and rolls items. `CollectionService` records discoveries. `QuestService` records participation and claims. Rules stay in services so several pages can use the same behavior.

JPA repositories load and save entities. Hibernate tracks managed changes and translates them into SQL. `WalletRepository.findByPlayerIdWithLock` requests a pessimistic lock. `EarnLootService` uses `JdbcTemplate` for JSON game state and `ON CONFLICT`. Both approaches share transaction boundaries.

Annotation	Meaning here
<code>@GetMapping</code> and <code>@PostMapping</code>	Map a URL and HTTP method
<code>@Valid</code>	Apply request constraints
<code>@Transactional</code>	Commit related changes together
<code>@Entity</code> and <code>@Table</code>	Map an object to a table
<code>@Version</code>	Detect stale entity updates
<code>@Value</code>	Read configuration

Transactions and proxies

`@Transactional` on a managed service creates a transaction around the call. Runtime exceptions roll it back. Calls between managed services join the transaction by default. A call to another method on this does not pass through Spring's proxy, so that method's annotation does not create an independent transaction. Keep critical entry points annotated and verify failure paths.

5 PostgreSQL and Flyway

Flyway versions the database structure. React never calls it, and it does not run a migration for every player request.

The startup sequence

Spring reads configuration and creates the DataSource. Flyway validates migration history and applies pending versioned SQL. Hibernate validates entity mappings because `spring.jpa.hibernate.ddl-auto=validate`. A missing table is a backend startup problem, not a CSS problem.

Files live in `src/main/resources/db/migration`. Flyway stores applied versions and checksums in `flyway_schema_history`. Editing an applied file can fail validation. Add a new version instead. Never reset a real database or use repair merely to hide a mismatch; determine which file changed and preserve data first.

Version	Main responsibility
V1	Identity, wallet, ledger, item schema
V2	Daily grants and seed items
V3	Rarity tiers and shop offers
V4	Owned crates and mystery rarity
V5	Collection and quest records
V6	Daily coins and saved card rounds
V7	Weapon designs
V8	Shop prices
V9	Account tutorial completion
V10	Recover still-owned discoveries

Constraints are the last boundary

Unique keys stop duplicate daily claims and collection rewards. Foreign keys attach items to real players and catalog entries. Wallet CHECK constraints prohibit negative balances. Service rules improve error messages; database constraints still defend integrity when requests overlap.

The backfill has a limit

V10 reconstructs discoveries from items that still exist in `inventory_items` and preserves their earliest acquisition timestamp. It cannot reconstruct items sold before discovery recording was fixed. Reliable historical records would be needed to recover those; do not invent them.

Read players for identity, wallets and `ledger_entries` for currency, `inventory_items` and `inventory_crates` for ownership, `player_collection` for discoveries, and `earn_rounds` for saved hands.

6 Coins ownership and concurrency

A gameplay mutation needs one transaction, a consistent locking order, and repeat-claim protection.

A shop purchase

ShopService loads the offer and checks its week. WalletService locks the buyer's wallet, checks affordability, subtracts the server price, and appends a negative ledger entry. The service creates the owned item and records discovery in the same transaction. A failure rolls back the debit and award together.

Consuming an item once

CrateService locks the player wallet before reading an owned crate or item. Two requests for that player wait on the same lock. After the first open consumes and awards, the second cannot find the crate. Selling uses the same boundary. Ownership queries include the player ID so another account cannot consume the item.

Claims and settlement

EarnLootService inserts a unique player-and-date claim with ON CONFLICT DO NOTHING. Only a successful insert credits coins. QuestService locks the wallet, checks completion, expiry, and claimed state, then marks the claim and credits the reward. Collection reward keys prevent repeated tier bonuses.

Failure	Protection
Overlapping balance writes	Wallet lock and entity version
Repeated daily claim	Unique date key and conditional credit
Repeated quest claim	Locked claimed flag
Concurrent crate open or sale	Lock before ownership lookup
Repeated deal	Same request ID returns saved round
Repeated stale action	Round version rejects it

Audit and retry limits

For a normal account, the sum of signed soft ledger entries should match its soft balance. balance_after records each change's result. Investigate a mismatch rather than editing an old entry. Developer wallets intentionally bypass normal spending and payouts, so exclude them from normal economy conclusions.

Card deals have request IDs and actions have versions. Shop and crate purchase POSTs do not yet have general purchase idempotency keys: repeating a buy can purchase another copy. The client does not blindly retry a lost network response. Inspect inventory and wallet before trying again. Purchase receipts and request IDs remain a follow-up before automatic buy retries are introduced.

7 The River from deal to reveal

The server owns the deck and outcome. React uses visible hand information to control atmosphere.

Deal and save

EarnLootService.start accepts a request UUID, game, and stake. It locks the wallet, returns a repeated request without charging twice, and rejects a second active hand. The stake must be between 10 and 100 percent of current coins. CardGameEngine deals the deck. The full state is persisted as JSON in earn_rounds.

The River starts with two player cards and three community cards against Nova and Atlas. Checking advances a street unless an AI raises. Raising commits more coins. After an AI raise, call or fold before continuing. At showdown, the engine compares the best five-card hands and settles the pot or split. Folding is valid; committed coins remain in the pot.

Recovery

GET /api/earn returns the saved round, so refreshing does not redraw it. Actions include the version the player saw. A stale version is rejected. A completed round is returned without paying again. The wallet lock serializes settlement and quest updates. The browser recovers authoritative state after a lost action response.

Atmosphere does not promise a win

High card and one pair stay calm. Two pair through full house use the original track when shaking activates. A flush draw requires a made two pair or better plus four suited cards. Flushes and full houses reach the strongest tier. Four of a kind or a straight flush changes music and scatters cards. Tension stays latched until the reveal, then music fades. These cues do not reveal hidden AI cards or calculate winning probability.

File	Responsibility
CardGameEngine.java	Rules and comparisons
EarnLootService.java	Persistence and settlement
pokerAtmosphere.js	Visible-hand intensity
RiverAtmosphere.jsx	Shake and audio lifecycle
HandFaithSequence.jsx	Stable scatter and phrases
ShowdownCutscape.jsx	Reveal and focus handling

Practice is separate

The tutorial uses scripted snapshots and zero stakes; it never starts a live round. Developer test deals require a server-validated TestIdentity and explicit dev profile. Production disables them. Hiding a React control is not the security boundary; the API rejects test parameters from regular players.

8 Progression odds and pacing

Use a dependable savings path as the baseline. Random returns and rare drops are not promised income.

The free daily loop

Signup provides 500 coins. Daily coins provide 500. Their completed quest adds 100. Opening three free daily boxes and claiming that quest adds another 500. The full free routine supplies 1,100 coins before optional sales, weekly goals, and one-time bonuses. These figures use the current default configuration.

The five MVP goals are daily coins, three completed hands, three opened boxes or crates, fifteen weekly hands, and fifteen weekly boxes or crates. None requires winning. Crafting and win-only goals are no longer newly assigned. Completed legacy goals remain claimable until expiry.

Shop tier	Coins	Days with daily coins only	Days with full free routine
Common	25	0	0
Basic	60	0	0
Excellent	150	0	0
Exotic	750	1	1
Extraordinary	12,000	23	11
Mystery	60,000	119	55

Read the assumptions

Estimates start with the signup bonus and assume no spending, wagering, or sales. The full free column includes the two free daily quests but excludes weekly and collection bonuses. These are saving days, not measured retention. Shop rotation can affect item availability. `scripts/balance-report.mjs` regenerates the baseline from source.

Tier bonuses are now Common 100, Basic 200, Excellent 300, Exotic 500, Extraordinary 6,000, and mystery 15,000. They are one-time rewards. Earlier payouts are not taken back. The revised values avoid cheap tier purchases generating rewards much larger than their total shop cost.

Honest odds

Normal crate odds come from CrateService. Daily odds are normalized from the same active catalog weights used for its roll. Both appear in the UI. Daily and normal openings do not currently enforce pity. The unused pity UI was removed rather than advertising a guarantee the opening code does not deliver.

Use observed play to decide whether endgame prices feel motivating or frustrating. Do not tune from a single lucky hand or assume every player claims every daily reward.

9. React: state, effects, and accessibility

React renders a description of the interface from current state. A state setter requests another render; it does not immediately change the variable inside an already running event handler. Keep server responses authoritative for balances and round outcomes. A client animation must never award coins.

An effect synchronizes with something outside rendering: fetching a saved round, subscribing to wallet events, or starting audio. Its cleanup must remove listeners, stop timers, and cancel animation frames. Development Strict Mode can run setup and cleanup again to expose mistakes. Duplicate visual setup should be harmless; reward claims belong in explicit user actions with backend protection.

Refs hold things that should survive rendering without triggering another render, such as an audio element or pending animation. A component key changes its identity. The tutorial intentionally resets each scripted preview with a key; real gameplay retains a round identity so cards do not scatter again on every action.

The first visit

OnboardingGate asks the server whether the account completed onboarding. The tutorial explains rewards, core modes, a no-stakes River preview, inventory, and one next action. Completion is saved on the account, not only in browser storage. Skipping is allowed and the footer can replay it. A failed completion save is shown for retry.

The preview uses five scripted snapshots: high card, two pair, flush potential, full house, and four of a kind. They demonstrate the atmosphere thresholds without calling the wager API. The winner animation labels itself practice and awards zero coins. Audio and intense motion are opt-in in the preview.

Usable beyond a mouse

Dialogs trap keyboard focus and make underlying content inert. Controls have labels; errors remain text, not only color. Sound and motion have separate persistent controls. System reduced-motion preference takes precedence over the app setting. CSS alone is insufficient: JavaScript animation loops and cutscene timing also respect reduced motion.

Lazy page imports split the initial download. Suspense displays a loading state while a page loads. This improves loading cost, but it does not replace handling API errors or expired sessions.

10. Build, diagnose, and release

Development and production are different

During development Vite serves React and proxies API requests to Spring Boot. In the release build, Maven's release profile copies the built frontend into the Boot JAR's static resources. SpaController forwards supported browser routes to index.html. React Router then selects the page. API routes remain backend routes.

Run `npm ci`, `npm run build`, and `npm run lint` in `lootvault-frontend`. Run Maven tests at the repository root using Java 21. Build the frontend before `mvn -Prelease package`. Check that the resulting JAR contains `static/index.html` and assets. The CI workflow repeats the build, regression scripts, backend checks, and publication safety scan.

A practical debugging sequence

1. Reproduce once and record the URL, action, response status, and time.
2. In Network, inspect the response body. A 401 means authentication is missing; a CSRF 403 means a request token is invalid; an origin rejection concerns configuration.
3. Match that request to the controller, then service, then repository query.
4. Inspect server logs without publishing passwords, cookies, or CSRF tokens.
5. Check ledger entries and persisted round state before deciding whether a retry is safe.
6. Fix the responsible layer and add a regression check for the behavior.

Do not retry purchases automatically after an uncertain network failure. Read the wallet and inventory first. Claims and round actions have protections described earlier, but arbitrary purchases do not yet have a durable client request identifier.

What has been exercised

Backend tests cover authentication, developer-mode gating, wallet behavior, and card-table security. Additional scripts exercise CSRF caching, atmosphere thresholds, pitch, and tutorial snapshots. A real local API smoke test checks racing daily and quest claims, crate opening, repeated deals, saved-round recovery, onboarding persistence, and logout. These checks are evidence for those scenarios, not proof of every production failure mode.

11. Deliberate interface design

The supplied UI article identifies decorative choices that often substitute for product decisions. For LootVault, the practical response is a consistent hierarchy: a clear page title, one useful next action, readable content, and restrained secondary actions.

Applying the seven warnings

Warning	Project response
Generic fonts	Use a readable system font for ordinary controls; reserve display treatment for game moments.
Automatic purple branding	Keep ordinary surfaces neutral. Purple in the River communicates a specific scattering effect.
Excessive rounded cards	Let introductions and progression text sit directly in the page; group only related interactive content.
Meaningless gradients	Remove lobby glow decoration and keep dramatic lighting tied to a hand state.
Random icons	Mode icons communicate cards, crates, and collection; decorative status dots were removed.
Unnecessary glow	Normal shop content stays calm. Winning and high-intensity gameplay supply the exceptional emphasis.
Motion without purpose	Button presses acknowledge input; navigation hover indicates action; intense motion has a control and a reason.

Hope means agency

Teach what a player can do next. Explain losses without humiliation. Offer recovery and learning rather than promising that persistence will win a wager. Folding is a valid decision. Quests reward participation and free openings rather than requiring wins. Practice shows the spectacle without risking a balance.

This release freezes scope around blackjack, poker, crates, daily boxes, shop, inventory, and simple quests. Unfinished banners and crafting are outside primary navigation. Adding a new mode should wait until these flows are understandable and reliable.

12. Work that needs people and deployment access

A small, unassisted playtest

Recruit 5-10 adult testers for a staging build with pretend coins. Use fresh accounts. Give each person about 20 minutes and these tasks without explaining how: sign in, claim free rewards, play a practice preview, start and finish a hand, buy and open a crate, find the item, understand a shop price, and return after refreshing an unfinished hand.

Observe silently first. Record time to first useful action, confusing labels, unsuccessful clicks, unexpected balances, errors, and the point where enjoyment drops. Afterward ask what the stakes and payouts meant, whether folding felt acceptable, and whether sound or motion controls were easy to find. Do not ask leading questions such as whether the interface was beautiful.

Use a simple issue sheet: task, expected behavior, observed behavior, severity, reproduction steps, and screenshot with private information removed. Fix blockers first, then repeat failed tasks. A useful launch target is at least 8 of 10 participants completing the core loop unaided, with no unexplained balance changes. That is a proposed acceptance criterion, not a result already measured.

Deployment checklist requiring your host

Choose the production domain and HTTPS host. Set private database environment variables there; never commit them. Developer login must remain disabled and its phrase empty in production. Configure exact allowed frontend origins if frontend and API use separate origins. Same-origin hosting simplifies cookies and CORS.

The production cookie is Secure, HttpOnly, and SameSite=Lax. Verify login, refresh, logout, expiration, and direct page navigation on the actual HTTPS domain. Start with one backend instance unless session storage is made shared; an in-memory session is lost on restart and is not automatically shared between replicas.

The local backup script has created a dump, restored it into a disposable database, and compared core table counts. This proves that local round trip. You still need an encrypted off-machine destination, retention policy, host credentials, scheduled backups, and a restore rehearsal from that destination. A backup stored beside the only database is not disaster recovery.

Remaining technical hardening

Before broad public launch, add durable purchase request IDs, login rate limiting, and staging browser tests against the deployed configuration. Decide how sessions survive scaling or accept that a restart requires signing in again. Persisted hands survive because they are in PostgreSQL. Monitor application errors and database capacity without exposing personal data. Verify asset licenses and audience suitability before publication.

Do not run a production migration without a current verified backup and a rollback plan. Flyway changes schema; it does not create your hosting provider's backups or automatically undo destructive SQL.

13. Learn by tracing and changing one behavior

Four exercises

1. Trace the daily coin claim from its button through `api/client.js`, `EarnController`, the service, wallet lock, ledger, and returned balance. Explain why two simultaneous requests pay once.
2. Change a tutorial sentence and rebuild. Then change a scripted hand and run `verify-onboarding.mjs`. Explain why neither operation changes a real wallet.
3. In a disposable database, add a small Flyway migration and observe the history row. Explain why editing an applied migration is unsafe.
4. Use the browser Network panel to compare a normal GET, a successful POST with CSRF, and a POST without the token. Identify which layer rejects the last request.

Write your explanation before asking an assistant to modify the code. Review the diff afterward and identify the source of truth for every changed behavior.

Primary references

These references explain framework behavior. The repository remains the source for LootVault-specific rules and prices.

- [React state](#)
- [React component identity and keys](#)
- [Spring Security CSRF](#)
- [Spring Boot database initialization](#)
- [Flyway validation and checksums](#)
- [Vite production builds](#)
- [Vite development proxy](#)
- [PostgreSQL 16 pg_dump](#)
- [PostgreSQL 16 pg_restore](#)